

Complex Engineering Project Report / Proposal

Bank Management System

Course Title: Introduction to Programming Language II (Java) Lab
Course Code: CSE282 **Section:** 4 **Semester:** Summer 2026
Department of Computer Science and Engineering
Submission Date: September 28, 2026

Team Members & GitHub Profiles

SL	Student Name	Student ID	GitHub Repository / Profile
1	Tasmeoul Miyad (Lead)	2024100000181	https://github.com/TasmeoulMiyad
2	Md Shawon Chowdhury Setu	2024100000238	https://github.com/shawonchowdhurysetu-arch
3	Md Farhatul Hasan Rabbi	2024100000411	https://github.com/farhatulhasan411
4	Miraj Masud Munna	2024100000521	https://github.com/Miraj28-ms
5	Rifat Ahamed	2024000000064	https://github.com/rifat-codes01

1. Introduction

The **Bank Management System** is a Java desktop application designed to model and automate standard banking operations. Built to address real-world financial workflows, it equips banking personnel with tools to open accounts, process deposits and withdrawals, perform atomic inter-account fund transfers, inspect ledgers, and manage persistent text records. The project implements core Object-Oriented Programming (OOP) paradigms (Encapsulation, Abstraction, Inheritance, and Polymorphism), custom structured exception handling, file persistence, and an interactive Java Swing graphical interface.

2. Objectives

2.1 Primary Aim

To design, implement, and validate an integrated Bank Management System demonstrating OOP architectural design, robust custom exception propagation, reliable file I/O operations, and event-driven GUI development.

2.2 Specific Project Goals

- Establish a class hierarchy with abstract superclasses and polymorphic method implementations.
- Implement complete **CRUD** operations (Create, Read, Update, Delete) on account records.
- Enforce financial safety limits (overdraft limits and minimum balance floors) via custom exceptions.
- Ensure persistent data management across application sessions using Java File I/O.
- Build an accessible Java Swing UI with integrated dialog handling and user feedback.

2.3 Key Functionalities

- **Create:** Register Savings and Current accounts with customer metadata and initial balances.
- **Read:** Query accounts via unique Account Numbers or search accounts by customer names.
- **Update:** Dynamically modify existing customer contact parameters (phone numbers and emails).
- **Delete:** Safely terminate account records with user confirmation dialogs.
- **Financial Operations:** Process balance-checked deposits, withdrawals, and inter-account transfers.
- **Auditing:** Log timestamped transactional activities with unique generated transaction IDs.

3. Motivation

Manual record keeping and spreadsheet tracking in financial settings often lead to input errors, security issues, and data discrepancies. Engineering a Bank Management System provides a realistic problem space where data integrity is critical:

- Banking requires strict invariants: an account must never drop below allowed limits, and funds must balance during transfers.
- The problem directly maps to the four core OOP pillars: Encapsulation guards balance data, Abstraction sets contract boundaries, and Polymorphism handles distinct account policies.
- It reinforces engineering fundamentals: input verification, exception propagation, and persistent file storage.

4. Applications

- **Deployment Contexts:** Local cooperative banks, credit unions, microfinance branches, and academic training environments.
- **Target Users:** Bank tellers, customer service representatives, branch accountants, and system auditors.
- **Practical Benefits:** Automates balance validations, lowers operational errors, avoids costly database setups via local flat-file storage, and speeds up daily ledger maintenance.

5. Project Description

5.1 Architecture Overview

The system uses a layered architecture separating visual presentation, coordination logic, and domain data models:

- **Presentation Layer (BankGUI):** Built with Java Swing (BorderLayout, action buttons, JOptionPane input/error dialogs, and a central JTextArea ledger).
- **Controller Layer (Bank):** Encapsulates the account collection, coordinates search/transfer routines, and interfaces with the disk storage engine.
- **Domain Model Layer:** Account (abstract superclass), SavingsAccount, CurrentAccount, Customer, and Transaction.
- **Exception Layer:** InsufficientFundsException and AccountNotFoundException.
- **Persistence Layer:** Text-based storage engine reading from and writing to `accounts.txt`.

5.2 Major Modules and Class Specifications

Class Name	Type / Layer	Primary Responsibilities
Account	Abstract Model	Defines base fields (<code>accountNumber</code> , <code>balance</code> , <code>customer</code> , <code>transactions</code>) and the abstract method <code>withdraw(double)</code> .
SavingsAccount	Subclass	Extends <code>Account</code> ; enforces minimum balance constraints during withdrawals.
CurrentAccount	Subclass	Extends <code>Account</code> ; provides overdraft ceiling protections during withdrawals.
Customer	Entity Model	Encapsulates customer identity data (<code>customerId</code> , <code>name</code> , <code>phone</code> , <code>email</code>).
Transaction	Entity Model	Creates timestamped transaction audit logs with auto-incremented IDs.
Bank	Controller	Manages account collections, executes fund transfers, and runs file read/write routines.
BankGUI	Presentation View	Provides the Swing GUI dashboard, action buttons, dialogs, and text area outputs.
Main	Entry Point	Launches the application on the Swing Event Dispatch Thread (EDT).

5.3 Workflow of the System

1. **Initialization:** On startup, `Main` opens `BankGUI`. The `Bank` controller scans `accounts.txt` and loads saved records into memory.
2. **User Interaction:** The user selects an operation from the GUI control panel. Input is captured and validated via modal dialogs.
3. **Core Execution:** The action is passed to `Bank`, which calls domain logic on the targeted `Account` objects.
4. **Exception Handling:** Rule violations throw custom exceptions, which are caught and displayed to the user via `JOptionPane` warning dialogs without crashing the system.
5. **Data Persistence:** On closing the GUI, an automated window-closing hook saves all current records back to `accounts.txt`.

5.4 OOP Concepts Mapping

- **Encapsulation:** All entity state variables are declared `private` and accessed or mutated exclusively through validated getters and setters.
- **Abstraction:** `Account` is an abstract base class that hides common balance logic while requiring concrete subclasses to implement `withdraw(double)`.
- **Inheritance:** `SavingsAccount` and `CurrentAccount` inherit state attributes and ledger behaviors from `Account`, reducing boilerplate code.
- **Polymorphism:** Subclasses override `withdraw(double)`, allowing the system to dynamically apply either minimum balance or overdraft limit rules at runtime.

6. Tools & Technologies

- **Programming Language:** Java Standard Edition (Java SE 17 / JDK 21)
- **Integrated Development Environment (IDE):** IntelliJ IDEA
- **Graphical User Interface:** Java Swing (`javax.swing.*`) and AWT (`java.awt.*`)
- **File Persistence:** Java I/O Streams (`PrintWriter`, `FileWriter`, `Scanner`) with delimiter-separated flat storage

- **Date/Time API:** `java.time.LocalDateTime` and `java.time.format.DateTimeFormatter`

7. Output / Results

7.1 System Outputs

- **Account Creation:** Generates and confirms new savings or current accounts, displaying success confirmations in the main terminal log.
- **Fund Transfer Log:** Outputs formatted receipts confirming transfers between valid sender and receiver accounts.
- **Transaction Ledger:** Formats audit trails with timestamps, unique transaction IDs, operation descriptions, and transacted amounts.

7.2 Error Handling Demonstrations

- **Minimum Balance Violation:** Triggers an `InsufficientFundsException`, opening a GUI dialog indicating the minimum balance constraint.
- **Overdraft Breach:** Displays an alert when a current account withdrawal exceeds the defined overdraft ceiling.
- **Account Not Found:** Throws an `AccountNotFoundException` on invalid account queries or transfers, informing the user via an error dialog.

8. Team Work Distribution & Contributions

Member Name	Modules / Deliverables Completed	OOP / Java Focus
Tasmeoul Miyad (Lead)	Core system integration, <code>Customer.java</code> , and abstract <code>Account.java</code> base.	Abstraction, Encapsulation, Architecture
Md Shawon Chowdhury Setu	Subclasses: <code>SavingsAccount.java</code> and <code>CurrentAccount.java</code> .	Inheritance, Method Overriding
Md Farhatul Hasan Rabbi	<code>Transaction.java</code> Classes, String Formatting, Date/Time API model and transaction history formatting.	Classes, String Formatting, Date/Time API
Miraj Masud Munna	Backend engine <code>Bank.java</code> and File I/O persistence (<code>accounts.txt</code>).	CRUD Operations, Collections, File Streams
Rifat Ahamed	<code>BankGUI.java</code> , <code>Main.java</code> , and custom exception classes.	Java Swing, Event Handling, Try-Catch

9. Conclusion

9.1 Project Summary

The Bank Management System delivers an interactive desktop solution for core financial operations. Combining a responsive Java Swing interface with an OOP architecture, the software ensures data integrity, safe transaction validation, and consistent file persistence across sessions.

9.2 Achievement of Objectives

All project objectives and rubric milestones were met:

- The class hierarchy demonstrates all four OOP pillars.
- Full CRUD features (Create, Read, Update, Delete) are operational.
- Business rules are enforced through custom exceptions and structured `try-catch` blocks.
- Account records persist reliably via file I/O operations.

9.3 Limitations and Future Improvements

- **Limitations:** Flat text file storage does not support transactional ACID rollback or concurrent multi-user write operations, and transaction histories reset between application sessions.
- **Future Improvements:** Transition to an embedded relational database (SQLite/MySQL via JDBC), add user authentication with role-based access control, and introduce automated interest calculation routines.